


lec 68 23Jan2024

14Nov 2024

Samarth Sir

React → JS library

- SPA CSEO ↓ faster ↑
- virtual DOM
- reuse of component
- component based architecture

→ types of components ⇒
① class → ye sena hai karna
② function → ye use karna hai

function → they are stateless
but hook ki

madad se
we can make it
stateful.

React CDN → ye copy kar to add in your file.

```
<script crossorigin src="https://unpkg.com/react@18/umd/
react.development.js"></script>
<script crossorigin src="https://unpkg.com/react-dom@18/umd/react-
dom.development.js"></script>
```

↳ In index.html footer

Let's use DOM first to understand DOM and React better :-

```
let rootEl = document.getElementById("root");  
let h1 = document.createElement("h1");  
h1.innerHTML = "hello h1 from DOM";  
rootEl.append(h1);
```

→ app.js

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>Document</title>  
</head>  
<body>  
  <div id="root"></div>  
  
  <script src="app.js"></script>  
</body>  
</html>
```



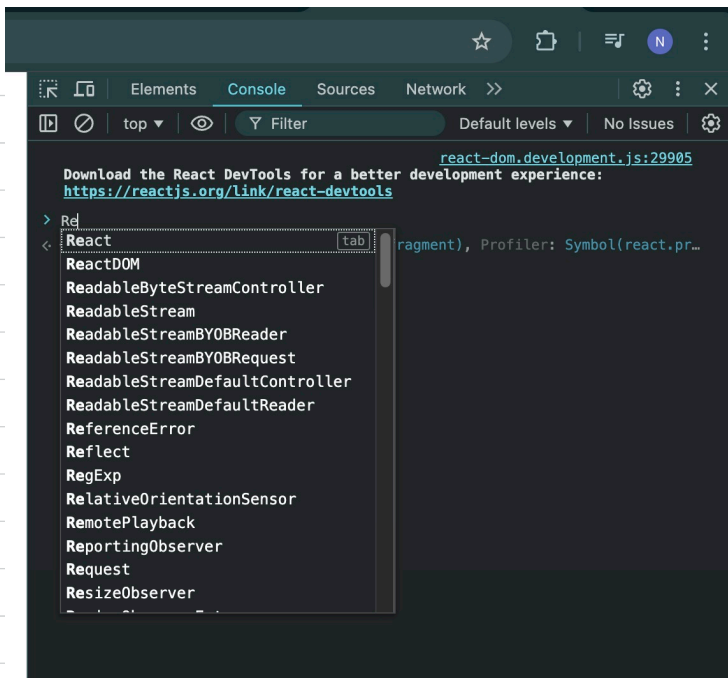
→ index.html



hello h1 from DOM

Now let's use react :-

React CDN code me use Karte hi, we can see use React related stuff in console.



Let's write the same code using React:-
html

Note:- Kyunki hum code app.js me likh rahe hai isliye "class" and "for" both are common in html and javascript. Isliye differentiation ke liye html ka class will be written as "className" and "for" as 'htmlFor'

```
let h1 = React.createElement(
  "h1",
  { id: "sam", className: "vohra" },
  "hello h1 from React"
);
```

we used className instead of class.

Syntax for creating element:-

React.createElement (" " , " " , " ") ;

↓
type of html element

↓
attributes like Id, class, etc

Now we will add this line in app.js :-

```
ReactDOM.createRoot(rootEl).render(h1);
```

Explanation

1. ReactDOM.createRoot(rootEl) initializes a React root at the `rootEl` element.
2. .render(h1) renders the `h1` React element you created into the DOM.

```
let rootEl = document.getElementById("root"); → l1  
  
// let h1 = React.createElement('h1', null, 'hello h1 from React')  
  
let h1 = React.createElement(  
  "h1",  
  { id: "sam", className: "vohra" },  
  "hello h1 from React"  
);  
  
ReactDOM.createRoot(rootEl).render(h1); → l4
```

→ app.js

write null when you don't have anything to write.

What is the difference b/w React, ReactDOM & ReactNative?

element create karega React and un elements ko dikhayega kon → ReactDOM.

agar apko web par React ki banayi hui chije dikhani hai → you use ReactDOM.

But agar apko same chij mobile application par dikhani hai, me use ReactNative.

React Native is a framework.

Explanation of 24:-

1. ReactDOM.createRoot(rootEl):

- `ReactDOM.createRoot()` is a function introduced in React 18 that initializes a **React root**. This root is the starting point for rendering and managing the React component tree.
- `rootEl` is the DOM element (the `<div>` with `id="root"`) that will contain your entire React app. By passing `rootEl` to `createRoot`, you tell React where in the DOM to place your components.
- **Why createRoot?** This new method allows React to use its latest features, including more advanced scheduling and concurrency for better performance and responsiveness.

2. .render(h1):

- The `.render()` function is called on the React root created by `createRoot()`.
- `render(h1)` tells React to render the `h1` element inside the `rootEl`.
- In this case, `h1` is a React element (created by `React.createElement`) that represents an `<h1>` HTML element with the text "hello h1 from React", along with some optional attributes like `id` and `className`.

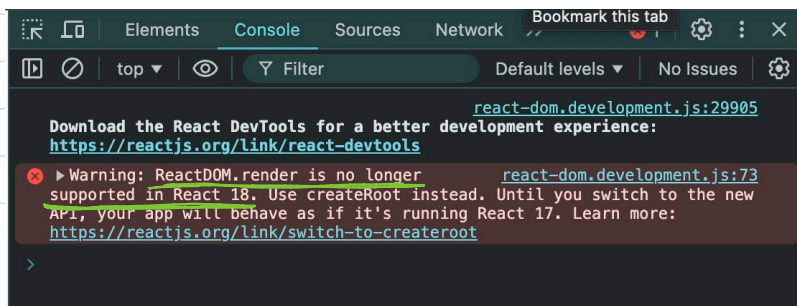
So, this line essentially mounts the `h1` element as a child of `rootEl` in the HTML, which causes it to display on the page.



or
use `ReactDOM.render(h1, rootEl);`

→ Render kar do h1 ko rootEl mei.

→ root element



⇒ warning aa gayi.

```
ReactDOM.createRoot(rootEl).render(h1);
```



pehle hamne
root banaya

then root ne render kiya h1 ko.

'ReactDOM.createRoot(rootEl).render(h1);' creates a React root on the 'rootEl' DOM element and renders the 'h1' React component inside it, displaying it on the page.

```
ReactDOM.createRoot(rootEl).render(h1, h2);
```



you cannot two
things, it only
accepts one.

```
ReactDOM.createRoot(rootEl).render([h1, h2]);
```



put elements
in array
Now it will work.

```

let rootEl = document.getElementById("root");

let h1 = React.createElement(
  "h1",
  { id: "sam", className: "vohra" },
  "hello h1 from React"
);

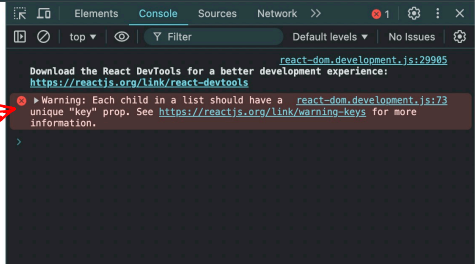
let h2 = React.createElement(
  "h2",
  null,
  "hello h2 from React"
);

// ReactDOM.createRoot(rootEl).render(h1, h2);
ReactDOM.createRoot(rootEl).render([h1, h2]);

```

hello h1 from React

hello h2 from React



Let's maybe root mei ab div ki nesting karni hai :- ^{element}

`<div id="root">`
~~`<div>`~~

`<h1>`
`<p>`

→ how will you do it?

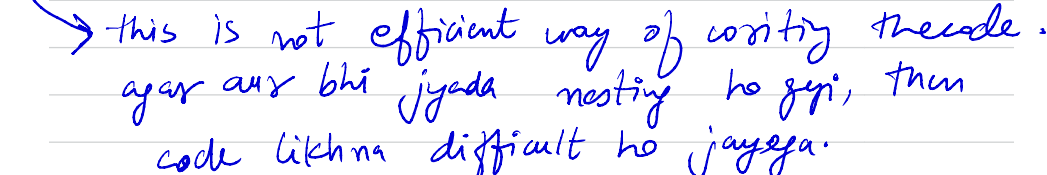
`</div>`

`</div>`

check code

hi i am div ke andar h1

hi i am div ke andar p



Now we will write HTML code directly inside app.js file. But Browser ko sirf JavaScript samajh aati hai, ~~so~~ iske liye ham babel use karenge jo JSX ko JS mei convert karega for browser.

Babel: compiler that converts JSX to pure JS (vanilla JS) then send it to browser.

Vanilla JS = Pure JS

Search "babel cdn" on google to get CDN link.

Babel.js.io

use babel cdn → paste in index.html

```
<script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>  
<script type="text/babel" src="app.js"></script>
```

→ add this in
"index.html"

```
<script src="app.js"></script>
```

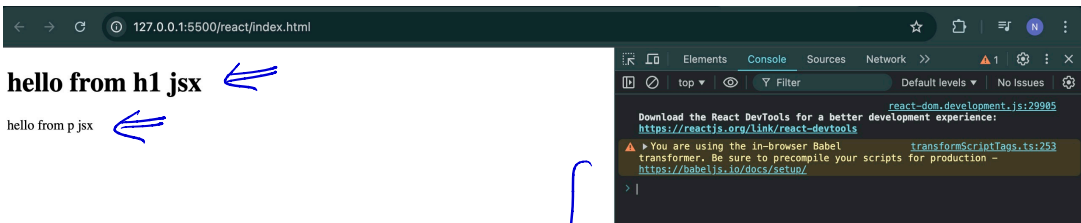
→ remove this

```
let rootEl = document.getElementById("root");
```

```
let div = (  
  <div>  
    <h1 id="sam">hello from h1 jsx</h1>  
    <p className="vohra">hello from p jsx</p>  
  </div>  
)
```

→ app.js

```
ReactDOM.createRoot(rootEl).render(div);
```



→ warning ki do not use
this babel in production.

Note:-

```
<script src="app.js"></script>
```

By default

type = "text/javascript" होता है
upar wale code me.

`<script src="app.js"></script>`

`<script type="text/javascript" src="app.js"></script>`

Both are same thing

if i am using jsx in my js file, then what is the use of

`type="text/babel"`
in

`<script type="text/babel" src="app.js"></script>?`

When you use `<script type="text/babel" src="app.js"></script>`, the `type="text/babel"` attribute tells the browser (with the help of Babel) that the script contains JSX or modern JavaScript syntax (ES6+) that needs to be transpiled into plain JavaScript before being executed.

hello from h1 jsx

hello from p jsx

The screenshot shows the Chrome DevTools Sources panel with the file `app.js` selected. The code is as follows:

```
55 // React.createElement(p, null, "hi i am div ke andar p"),
56 // });
57
58 // ReactDOM.createRoot(rootEl).render(div);
59
60 // 4..
61
62 // JSX
63 let rootEl = document.getElementById("root");
64
65 let div = {
66   <div>
67     <h1 id="sam">hello from h1 jsx</h1>
68     <p className="vohra">hello from p jsx</p>
69   </div>
70 };
71
72 ReactDOM.createRoot(rootEl).render(div);
73
```

Handwritten annotations include a green box around the `app.js` file in the file explorer, a green box around the JSX code, and the text "this is my code" with an arrow pointing to the boxed code. Another arrow points to the bottom of the panel with the text "Now click here".

The screenshot shows the same file `app.js` but now displaying the transpiled JavaScript code:

```
51 // let rootEl = document.getElementById("root");
52
53 // let div = React.createElement("div", null, [
54 //   React.createElement("h1", null, "hi i am div ke andar h1"),
55 //   React.createElement("p", null, "hi i am div ke andar p"),
56 // ]);
57
58 // ReactDOM.createRoot(rootEl).render(div);
59
60 // 4..
61
62 // JSX
63 var rootEl = document.getElementById("root");
64 var div = /*#__PURE__*/React.createElement("div", null, /*#__PURE__*/
65   [
66     {
67       id: "sam",
68     }, "hello from h1 jsx"), /*#__PURE__*/React.createElement("p", {
69       className: "vohra",
70     }, "hello from p jsx");
71   ],
72 );
73 ReactDOM.createRoot(rootEl).render(div);
74
```

Handwritten annotations include a green box around the transpiled code and the text "this is converted code" with an arrow pointing to the boxed code. The status bar at the bottom indicates "Line 61, Column 1" and "Coverage: n/a".

⇒ the code is converted to Vanilla JS.

(58:42)

ek folder mei

Maanlo tumhare paas n multiple JS files hai but browser ko to ek hi JS file bhej sakta hai. Also there will be multiple CSS files corresponding to your JS files. Now bundler comes into role.

A bundler is a tool that takes all your JavaScript code, along with any other assets like CSS or images, and combines them into a single file (or a few files) that can be served to the browser.

There are many bundlers in the market :-

- ↳ web packs
- ↳ vite
- ↳ parcel

jab hum create react app ki maddad se application banate hai, then web packs is the default bundler.

vite, parcel $\Rightarrow$ they are nice bundlers.

web packs $\rightarrow$ not a good bundler.

(1:25:04) break over:-

Search "legacy react" on google, then open the docs, then click on "create a new react app".

This site is no longer updated. [Go to react.dev](#)

React

[Docs](#) [Tutorial](#) [Blog](#) [Community](#)

Search

v18.2.0 Languages [GitHub](#)

Create a New React App

These docs are old and won't be updated. Go to [react.dev](#) for the new React docs. See [Start a New React Project](#) for the recommended ways to create an app.

Use an integrated toolchain for the best user and developer experience.

This page describes a few popular React toolchains which help with tasks like:

- Scaling to many files and components.
- Using third-party libraries from npm.
- Detecting common mistakes early.

INSTALLATION ^

Getting Started

Add React to a Website

Create a New React App

CDN Links

Release Channels

MAIN CONCEPTS v

ADVANCED GUIDES v

API REFERENCE v

HOOKS v

TESTING v

This site is no longer updated. [Go to react.dev](#)

React

[Docs](#) [Tutorial](#) [Blog](#) [Community](#)

Search

v18.2.0 Languages [GitHub](#)

Create React App

Create **React App** is a comfortable environment for **learning React**, and is the best way to start building a **new single-page application** in React.

It sets up your development environment so that you can use the latest JavaScript features, provides a nice developer experience, and optimizes your app for production. You'll need to have **Node >= 14.0.0** and **npm >= 5.6** on your machine. To create a project, run:

```
npx create-react-app my-app
cd my-app
npm start
```

Note
npx on the first line is not a typo — it's a package runner tool that comes with npm 5.2+.

Create React App doesn't handle backend logic or databases; it just creates a frontend build

INSTALLATION ^

Getting Started

Add React to a Website

Create a New React App

CDN Links

Release Channels

MAIN CONCEPTS v

ADVANCED GUIDES v

API REFERENCE v

HOOKS v

TESTING v

`npx` `create-react-app` `react-basic`

↙ ↘

↘ folder name
(only small letters are allowed when naming folder)

The command `npx create-react-app my-app` is used to create a new React application called `my-app` in a directory of the same name

→ this command is too much time taking. (around/min)
it was done in less 68 folder in terminal of VS code.

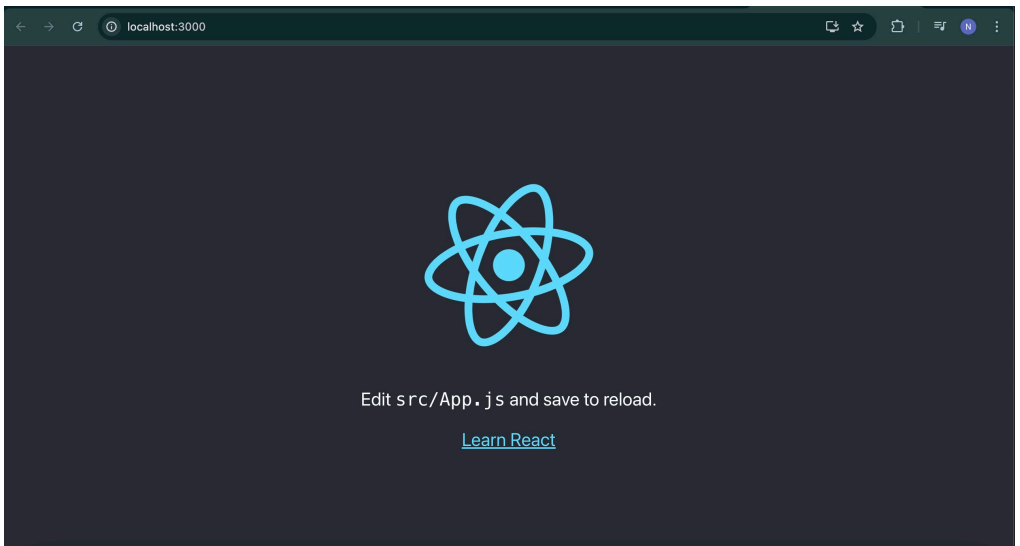
cd react-basic → ye likho in lec 68 folder

then npm start likho.

↓

ye likhte hi browser mei
default react app khul jayegi.

↓



folder "react-basic" ke andar src & public folder honge.

public folder mei index.html hai. Hamne bhi apne code mei div ko root id di thi, yaha par inhone bhi yahi di hai.



```
react-basic > public > < index.html > < html > < body > < div#root
2 <html lang="en">
3 <head>
8 <meta
9   name="description"
10  content="Web site created using create-react-app"
11 />
12 <link rel="apple-touch-icon" href="%PUBLIC_URL%/logo192.png" />
13
14 <link rel="manifest" href="%PUBLIC_URL%/manifest.json" />
15
16 <title>React App</title>
17 </head>
18 <body>
19   <noscript>You need to enable JavaScript to run this app.</noscript>
20
21   <div id="root"></div>
22 </body>
23 </html>
24
25
```

we only have one .html file in our react application. Kyunki ~~se~~ react mei SPA bante hai isliye ek hi file hai html ki kyunki react application sirf ek hi baar chalti hai as react makes single page application.

src folder se sirf ne 4 files delete karra di:-

App.test.js
logo.svg
reportWebVitals.js
setupTests.js

} → delete kar do
⇓
optional hai

src ke index.js mei se web vitals wala code ki do lines uda do.

App.css, index.css, $\Rightarrow$ content uda do $\rightarrow$ we don't need it.

App.js

(1:52:53)

Hamare पास itni saari files hai, inme se main file konsi hai?

$\rightarrow$ do not forget this hierarchy

```
graph LR; cat.js --> App.js; dog.js --> App.js; honey.js --> App.js; App.js --> index.js; index.js --> index.html;
```

ye components hai $\rightarrow$ we discussed in beginning of React lecture.

Code mei components ka jayega App.js $\rightarrow$ then App.js commands bhejegi index.js ko, then index.js commands bhejegi index.html ko.

main file $\rightarrow$ index.js

App.js $\rightarrow$ ye component hai

npm create-react-app se mei babel and webpaks use hota hai. you can check the same in ~~one~~ node-modules folder.

we can see node-modules mei kayi saare modules hai isliye kaafi time laga tha create-react-app mei.

Ab hum vite.js use karenge instead of webpaks since webpaks is inefficient.

vitejs.dev → documentation

npm create vite@latest → put this in VS code terminal
↓
then select React

↓
then select JavaScript

↓
cd react-vite-basic

↓
npm install

npm run dev

<http://localhost:5173/>

→ now go to
you can see
vite is working

vite is comparatively faster than webpacks.

